ENERGY ENGINEER FUNDAMENTAL KNOWLEDGE DEVELOPMENT

Accelerated 3-Month Learning Plan

A 12-week roadmap updated after Session 7: the battery dispatch, optimization, degradation, multi-day analysis, and real-carbon pipeline are already substantially beyond the original Week 1 scope.

7-10 hours per week	**One coherent microgrid project**	**Evidence-based completion gates**

Program objective

Build independent working proficiency across Python, pandas, optimization, real grid data, OpenDSS, SQL, Git, SPICE, C++, and Arduino. Each tool extends the same microgrid workflow rather than becoming a disconnected tutorial.

MONTH 1	MONTH 2	MONTH 3
Close the data loop; validate dispatch in OpenDSS	Data systems, circuits, and robust control	Network-aware control, compiled code, embedded work, integration

SEQUENCING DECISION

Consolidate, then advance the physics

- Week 2 closes Session 7: weighted-carbon analysis, strict API validation, real CAISO prices, a real-price-plus-real-carbon comparison, cleanup, tests, and a short technical recap.
- OpenDSS moves forward from Weeks 7-8 to Weeks 3-4 because the optimizer interface and dispatch schedules already exist.
- SQL and SPICE remain in the program, but move behind the first software-to-power-flow integration so the current learning momentum is preserved.
- Network-aware optimization follows after foundational power flow and operational robustness, avoiding premature constraint design.

PROGRAM OVERVIEW

Revised 12-week roadmap

WEEK	FOCUS	PRIMARY OUTCOME
1	Dispatch and optimization foundation	Completed core model through multi-day optimization; Session 7 integration is substantially underway.
2	Real-data consolidation and software gate	A validated real-carbon/real-price analysis pipeline and clean Week 1 technical package.
3	OpenDSS fundamentals	A radial microgrid feeder with interpretable voltage, loading, loss, and base-case checks.
4	OpenDSS with DER dispatch	Electrical validation of rule-based and optimized 15-minute schedules.
5	SQL, provenance, and reproducibility	A relational experiment store and reproducible queries linked to dispatch runs.
6	SPICE fundamentals	Verified analytical and simulated circuit behavior.
7	SPICE for power electronics	A converter model with parameter-sensitivity analysis.
8	Rolling control and operational robustness	A receding-horizon controller with forecast errors and safe fallbacks.
9	Network-aware dispatch	Dispatch constraints or corrective logic informed by feeder limits.
10	C++ fundamentals	A compiled battery/load simulator mirroring the Python model.
11	Arduino and embedded monitoring	A sensor, ADC, serial-logging, and simple control experiment.
12	Integrated capstone	A documented price-, carbon-, and network-aware microgrid controller study.

CURRENT COMPLETION GATE

Do not start OpenDSS until Session 7 closes with: power-weighted carbon metrics; duplicate/missing/granularity and merge-size checks; real CAISO price ingestion; real carbon + real price optimization; consistent baseline evaluation; cleaned code/tests; and a concise Week 1 recap. Multi-day real-carbon ingestion may be completed here or carried as a bounded Week 5 data task.

WEEKS 1-2

Foundation status and consolidation

Week 1 exceeded its original scope. Week 2 is therefore a closure sprint, not another broad software-foundations week.

WEEK 1

Dispatch and optimization foundation
Status: approximately 85-90% complete after Session 7 work to date.

CORE WORK	WEEKLY DELIVERABLE
● Battery timestep model, simulation, tests, and 15-minute pandas inputs. ● Rule-based price/carbon dispatch with cost, emissions, SOC, and plots. ● CVXPY cost, carbon, and combined objectives. ● Degradation cost, throughput/EFC, sensitivities, and validation. ● Multi-day horizons, SOC continuity, daily-reset comparison, and reusable checks. ● Electricity Maps historical carbon, timezone conversion, merge, and fair schedule comparison.	Evidence package showing completed Sessions 1-6 and the current Session 7 result, including the observed cleaner-charge/dirtier-discharge behavior and approximately 0.35% real-carbon emissions improvement.
WHY NOW	This page records actual progress instead of asking the learner to repeat already-completed work.

WEEK 2

Real-data consolidation and software gate
Finish Session 7 and make the optimizer-to-power-flow handoff trustworthy.

CORE WORK	WEEKLY DELIVERABLE
● Compute energy/power-weighted charging and discharging carbon metrics. ● Validate missing, duplicate, unordered, timezone, granularity, unit, and merge-length conditions. ● Ingest real CAISO price data through GridStatus; preserve provenance and cache/replay inputs. ● Run synthetic, real-carbon, real-price, and combined real-signal comparisons on common baselines. ● Refactor interfaces, clean solver noise, add tests, configuration, README, and final Week 1 recap.	A reproducible one-day real-signal pipeline plus a clean scenario runner, validated metrics table, plots, tests, and an OpenDSS handoff file containing timestamp, load, PV, battery charge/discharge, grid exchange, and SOC.
WHY NOW	A short closure week prevents data defects or ambiguous sign conventions from contaminating every later power-flow result.

WEEKS 3-4

Month 1 - Electrical feasibility arrives early

With stable 15-minute schedules available, the next pedagogical step is to test whether economically attractive operation is physically acceptable.

WEEK 3

OpenDSS fundamentals
Move from an energy-balance model to distribution-system power flow.

CORE WORK	WEEKLY DELIVERABLE
● Circuit, Vsource, Transformer, Line, Load, Bus, voltage bases, and solution modes. ● Per-unit voltage, loading, feeder losses, radial voltage drop, and sign conventions. ● Build a small grid-connected feeder with a point of common coupling, transformer, line, load bus, PV, and storage location. ● Check a base case against hand estimates and explain each major parameter.	A documented radial feeder with a one-line diagram, acceptable base case, and interpretable voltage, loading, loss, and grid-exchange results.
WHY NOW	The learner now has enough software maturity to focus on electrical meaning rather than debugging the dispatch model at the same time.

WEEK 4

OpenDSS with PV, storage, and optimized dispatch
Replay Python schedules at 15-minute resolution and identify infeasible intervals.

CORE WORK	WEEKLY DELIVERABLE
● Use LoadShape, PVSystem, Storage or controlled injections, and daily solution mode. ● Map units, timestamps, phases, charge/discharge signs, and grid import/export consistently. ● Compare no-battery, rule-based, cost-optimal, carbon-optimal, and combined schedules. ● Measure voltage profile, transformer/line loading, losses, reverse flow, and violations. ● Define a machine-readable feasibility report for later control logic.	A 24-hour validation report that links each schedule to electrical outcomes and flags when energy-optimal dispatch is physically unacceptable.
WHY NOW	This realizes the original PDF's key bridge earlier, because the required upstream schedules already exist.

WEEKS 5-7

Month 2 - Data traceability and circuit intuition

WEEK 5

SQL, provenance, and reproducibility
Persist inputs, simulation runs, constraints, and outputs after the first full integration.

CORE WORK	WEEKLY DELIVERABLE
● Tables for site, measurement, signal provenance, simulation_run, dispatch_result, and powerflow_result. ● SELECT, WHERE, ORDER BY, GROUP BY, HAVING, JOIN, keys, NULLs, indexes, and normalization. ● Load pandas results into SQLite; query cost, emissions, cycling, violations, and worst intervals. ● Record configuration and source metadata sufficient to reproduce a run.	A relational database containing multiple scenarios and at least ten engineering queries, including joined dispatch/power-flow diagnostics.
WHY NOW	SQL is more meaningful after both optimizer and feeder results exist; the schema can now reflect real analytical relationships.

WEEK 6

SPICE fundamentals
Re-establish analytical checks before trusting circuit simulation.

CORE WORK	WEEKLY DELIVERABLE
● Reference nodes, current/voltage measurement, DC operating point, transient analysis, AC sweep, and parameter sweep. ● Resistive divider, RC low-pass, RL, and RLC exercises. ● Compare calculations, limiting behavior, and simulated plots.	Three to four verified simulations with concise calculation-versus-simulation explanations.
WHY NOW	This develops component-level judgment without interrupting the earlier microgrid software-to-feeder momentum.

WEEK 7

SPICE for power electronics
Connect switching behavior to DER interfaces.

CORE WORK	WEEKLY DELIVERABLE
● Diodes, MOSFETs, PWM, duty cycle, switching frequency, inductors, capacitors, and non-ideal trends. ● Build a buck converter and verify the ideal duty-ratio relationship. ● Sweep load, L, C, duty cycle, and switching frequency; measure ripple and transients.	A converter model and parameter-sensitivity report explaining observed trends and model limitations.
WHY NOW	Converter intuition prepares the learner to interpret device limits without pretending the OpenDSS model resolves switching physics.

WEEKS 8-9

Month 2 continued - From validation to control

WEEK 8

Rolling optimization and operational robustness
Turn perfect-forecast scheduling into a simplified controller.

CORE WORK	WEEKLY DELIVERABLE
● Receding-horizon workflow with planned versus realized SOC. ● Load/PV forecast errors and delayed, missing, stale, or invalid price/carbon signals. ● Safe fallback dispatch, clipping, operator override, and execution confirmation. ● Track cost, emissions, peaks, cycling, and constraint violations under failure cases.	A 15-minute rolling-controller simulation with at least three failure scenarios and documented fallback behavior.
WHY NOW	Robust execution logic should exist before feeder limits are allowed to influence automated decisions.

WEEK 9

Network-aware dispatch
Use power-flow evidence to prevent or correct infeasible schedules.

CORE WORK	WEEKLY DELIVERABLE
● Translate transformer, import/export, and selected voltage/thermal findings into tractable limits or corrective rules. ● Compare sequential optimize-then-check, iterative correction, and simplified constraint approaches. ● Re-run cost, carbon, and combined cases; quantify lost objective value versus improved feasibility. ● Document which limits are exact, approximated, or validated only after dispatch.	A network-aware schedule comparison showing feasibility, objective tradeoffs, residual violations, and modeling assumptions.
WHY NOW	Power-flow literacy and operational fallbacks now exist, so network constraints can be introduced with defensible meaning.

WEEKS 10-12

Month 3 - Implementation and integration

WEEK 10

C++ fundamentals through model translation
Learn compiled engineering programming by porting familiar battery logic.

CORE WORK	WEEKLY DELIVERABLE
● Variables, loops, functions, vectors, references, structs/classes, compilation, and basic error interpretation. ● Port battery energy update, validation, and full-day iteration from Python. ● Compare outputs and edge cases against the Python reference.	A compiled battery/load simulator with automated or documented parity checks against Python.
WHY NOW	Familiar physics keeps attention on compiled-language concepts.

WEEK 11

Arduino and embedded monitoring
Connect software concepts to measured signals and simple safe control.

CORE WORK	WEEKLY DELIVERABLE
● Digital/analog I/O, PWM, serial communication, and sampling intervals. ● ADC range/resolution, calibration, noise, filtering, timestamps, and units. ● Log a low-risk sensor to Python and apply threshold or proportional output logic.	A sensor-to-serial-to-Python experiment with calibration notes and simple control behavior.
WHY NOW	The goal is disciplined measurement and command handling, not high-power hardware.

WEEK 12

Integrated capstone
Combine real signals, optimization, robustness, storage, and feeder validation.

CORE WORK	WEEKLY DELIVERABLE
● Replay validated load, PV, price, and carbon inputs with provenance. ● Compare no-battery, price, carbon, combined, and network-aware strategies. ● Run rolling/failure scenarios and OpenDSS validation. ● Package code, database, configuration, tests, results, diagrams, README, limitations, and a concise report.	A reproducible price-, carbon-, and network-aware microgrid controller study suitable for portfolio review.
WHY NOW	Success is coherent integration and engineering judgment, not maximum algorithmic complexity.

COMPLETION GATES

Evidence required before advancing

GATE	PASS CONDITION
Gate A - Start Week 2	Sessions 1-6 completed; Session 7 real-carbon workflow demonstrated; unresolved tasks listed and bounded.
Gate B - Start OpenDSS	Real price and carbon merged; common-baseline comparisons complete; data checks pass; handoff columns, units, timezone, and signs documented.
Gate C - Finish Month 1	Base feeder validated; all dispatch scenarios replayed; voltage/loading/loss/reverse-flow results reported; infeasible intervals identified.
Gate D - Start network-aware control	Rolling controller has fallbacks; feeder constraints have clear physical source and units; baseline feasibility results are reproducible.
Gate E - Capstone complete	Code/tests/configuration/data provenance/database/results/limitations are packaged; another reader can reproduce at least one full scenario.

Progress dashboard

COMPLETE	Battery model; 15-minute data; rules; CVXPY objectives; degradation; multi-day SOC continuity; core validation.
SUBSTANTIALLY COMPLETE	Electricity Maps carbon ingestion, timezone conversion, optimizer merge, common real-carbon evaluation, dispatch interpretation.
NEXT	Weighted analysis; hardened data checks; GridStatus/CAISO prices; combined real-signal run; cleanup and recap.
THEN	OpenDSS feeder fundamentals and 15-minute schedule validation.

WEEK 12

Capstone architecture

INPUTS	DATA SYSTEM	DECISION	VALIDATION
Load, PV, price, carbon, configuration	Validation, replay/cache, pandas, SQL, provenance	Rules or CVXPY; rolling horizon; fallbacks; network-aware limits	OpenDSS voltage, loading, losses, reverse flow, violations

Required scenario comparison

SCENARIO	PURPOSE	KEY QUESTION
A. No battery	Baseline	What happens without storage?
B. Price-based	Cost response	Do savings change emissions, peaks, or feasibility?
C. Carbon-based	Emissions response	What cost and electrical impacts accompany lower emissions?
D. Combined	Tradeoff	What schedule is defensible under explicit weights?
E. Network-aware	Feasibility	What objective value is exchanged for physical acceptability?

Metrics

ECONOMIC	ENVIRONMENTAL	OPERATIONAL	ELECTRICAL
Energy cost, export revenue, demand charge, cycling/degradation cost	Grid-import emissions and carbon shifted across time	SOC, throughput, EFC, peaks, fallback use, forecast error, command failures	Voltage, loading, losses, reverse flow, import/export, violations

END STATE

Three-month success criteria

The program succeeds when the learner can independently build, test, explain, and reproduce the full workflow without relying on a tutorial for every step.

1	Validated real or replayed input signals with provenance.
2	Python/pandas analysis and SQL storage.
3	Battery rules, optimization, degradation, and robust rolling control.
4	OpenDSS electrical validation and network-aware corrective logic.
5	Scenario metrics, plots, explanations, tests, and limitations.

TOOL / AREA	EXPECTED CAPABILITY
Python and pandas	Build tested models, validate time series, calculate metrics, and create reproducible analyses.
Optimization	Formulate objectives and constraints, detect unrealistic behavior, and explain tradeoffs.
Real data, SQL, and Git	Acquire, validate, cache, store, query, version, and document inputs and results.
SPICE	Analyze circuits and switching behavior against calculations and physical expectations.
OpenDSS	Build feeders, replay DER schedules, diagnose violations, and inform dispatch constraints.
C++	Compile a basic engineering simulation and verify parity with a reference model.
Arduino	Acquire a sensor signal, understand ADC/PWM limits, log data, and apply safe control.

End goal: one coherent energy-engineering workflow supported by tested code, real signals, physical validation, safe control behavior, and clear technical communication.